

Subject :- Cyber Security

Practical 4

Aim:- Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

CODE:-

```
import hashlib
import math
```

```
# Calculate GCD
```

```
def gcd(a, b):
```

```
    while b != 0:
```

```
        a, b = b, a % b
```

```
    return a
```

```
# Calculate modular inverse
```

```
def mod_inverse(e, phi):
```

```
    for d in range(1, phi):
```

```
        if (e * d) % phi == 1:
```

```
            return d
```

```
    return None
```

```
# Generate RSA keys
```

```
def generate_keys(p, q):
```

```
    n = p * q
```

```
    phi = (p - 1) * (q - 1)
```

Yash bhagat

T072

```
e = 2
```

```
while e < phi:
```

```
    if gcd(e, phi) == 1:
```

```
        break
```

```
    e += 1
```

```
d = mod_inverse(e, phi)
```

```
public_key = (e, n)
```

```
private_key = (d, n)
```

```
return public_key, private_key
```

```
# Generate hash of message
```

```
def hash_message(message):
```

```
    hash_value = hashlib.sha256(message.encode()).hexdigest()
```

```
    return int(hash_value, 16)
```

```
# Create digital signature using private key
```

```
def sign_message(message, private_key):
```

```
    d, n = private_key
```

```
    hash_value = hash_message(message)
```

```
# Reduce hash for this small educational RSA example
```

```
    hash_value = hash_value % n
```

```
    signature = pow(hash_value, d, n)
```

Yash bhagat

T072

```
return signature
```

```
# Verify digital signature using public key
```

```
def verify_signature(message, signature, public_key):
```

```
    e, n = public_key
```

```
    hash_value = hash_message(message)
```

```
    hash_value = hash_value % n
```

```
    decrypted_signature = pow(signature, e, n)
```

```
    return decrypted_signature == hash_value
```

```
# Main program
```

```
def main():
```

```
    print("=====")
```

```
    print("    RSA DIGITAL SIGNATURE")
```

```
    print("=====")
```

```
# Small prime numbers for demonstration
```

```
p = 61
```

```
q = 53
```

```
public_key, private_key = generate_keys(p, q)
```

```
print("\nRSA Keys Generated")
```

```
print("Public Key :", public_key)
```

Yash bhagat

T072

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

```
print("Private Key:", private_key)

# Message
message = input("\nEnter message: ")

# Create signature
signature = sign_message(message, private_key)

print("\nOriginal Message:", message)
print("Digital Signature:", signature)

# Verify original message
result = verify_signature(message, signature, public_key)

print("\nVerification of Original Message:", end=" ")

if result:
    print("VALID")
    print("Message integrity and authenticity verified.")
else:
    print("INVALID")

# Test modified message
modified_message = input(
    "\nEnter modified message to test verification: "
)

modified_result = verify_signature(
    modified_message,
    signature,
    public_key
```

)

```
print("\nVerification of Modified Message:", end=" ")
```

```
if modified_result:
```

```
    print("VALID")
```

```
else:
```

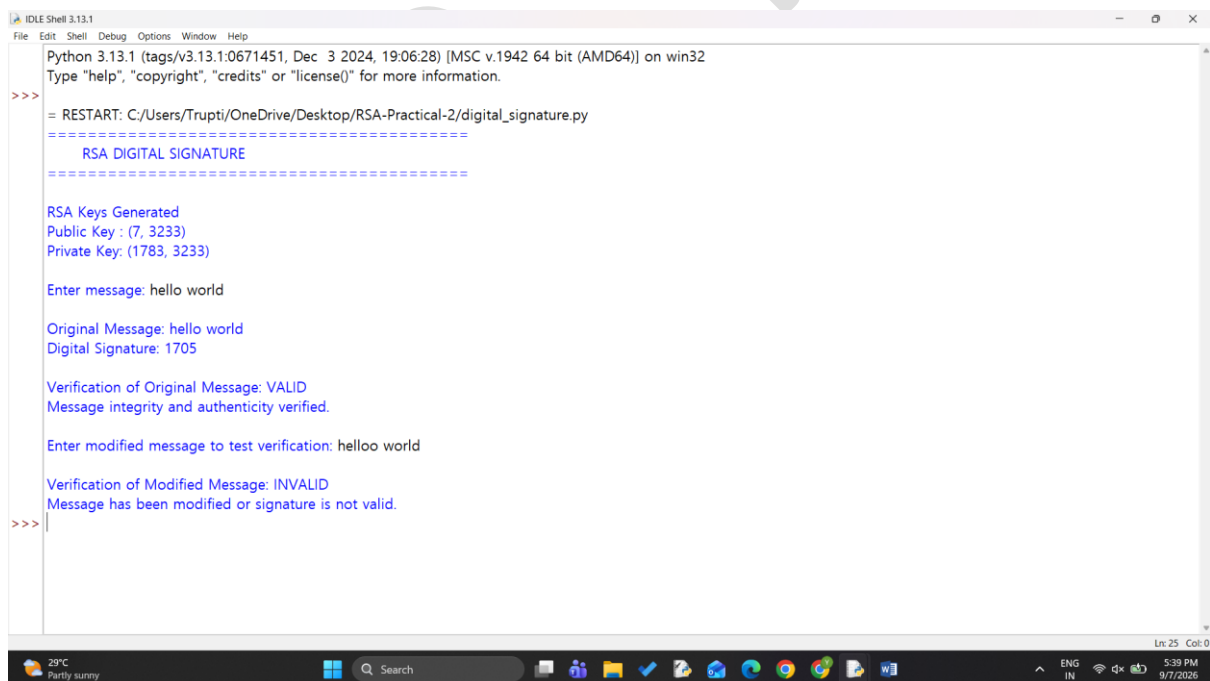
```
    print("INVALID")
```

```
    print("Message has been modified or signature is not valid.")
```

```
if __name__ == "__main__":
```

```
    main()
```

output:-



```
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Trupti/OneDrive/Desktop/RSA-Practical-2/digital_signature.py
=====
RSA DIGITAL SIGNATURE
=====

RSA Keys Generated
Public Key : (7, 3233)
Private Key: (1783, 3233)

Enter message: hello world

Original Message: hello world
Digital Signature: 1705

Verification of Original Message: VALID
Message integrity and authenticity verified.

Enter modified message to test verification: helloo world

Verification of Modified Message: INVALID
Message has been modified or signature is not valid.
>>>
```